```
----------------------------------------------------------------------------------------------------------------------¶-- CC BY SA   EXPANDER.DECLAR
ATIONS.TYPES_DECLS.ADB   VINCENT MORIN¬21/11/2025 UNIVERSITE DE BRETAGNE OCCIDENTALE¶------------------------------------------------------------
------------------------------------------------------¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬0¬1¶¶¶separate ( EXPANDER.DECLARATIONS )¶¬¬¬¬------------¶package body¬¬¬T
YPES_DECLS¶¬¬¬¬------------¶is¶¶    procedure CODE_ENUMERATION_DECL¬¬( TYPE_DECL :TREE );¶    procedure CODE_INTEGER_DECL¬¬( TYPE_DECL :TREE );¶    pro
cedure CODE_FIXED_DECL¬¬¬( TYPE_DECL :TREE );¶    procedure CODE_FLOAT_DECL¬¬¬( TYPE_DECL :TREE );¶¶    procedure CODE_RECORD_DECL¬¬( TYPE_DECL :TREE
);¶    procedure CODE_UNCONSTRAINED_ARRAY_DECL¬( TYPE_DECL :TREE );¶    procedure CODE_CONSTRAINED_ARRAY_DECL¬( TYPE_DECL :TREE );¶    procedure CODE_
ACCESS_DECL¬¬( TYPE_DECL :TREE );¶¶¶¶¬¬¬--==============--¶  procedure¬¬  CODE_TYPE_DECL¬¬( TYPE_DECL :TREE )¶  is¬¬¬--==============--¶¶    TYPE_NAME
¬: TREE¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶    TYPE_SPEC¬: TREE¬:= D( SM_TYPE_SPEC,¬TYPE_NAME¬);¶  begin¶¬¬¬¬¬-- SCALAR¬TYPES¶¶    if¬ TYPE_SPEC.TY = D
N_ENUMERATION¬then  CODE_ENUMERATION_DECL¬      ( TYPE_DECL );¶    elsif¬ TYPE_SPEC.TY = DN_INTEGER¬¬then  CODE_INTEGER_DECL¬      ( TYPE_DECL );¶    el
sif¬ TYPE_SPEC.TY = DN_FIXED¬¬then  CODE_FIXED_DECL¬      ( TYPE_DECL );¶    elsif¬ TYPE_SPEC.TY = DN_FLOAT¬¬then  CODE_FLOAT_DECL¬      ( TYPE_DECL );¶
¶    elsif¬ TYPE_SPEC.TY = DN_RECORD¬¬then  CODE_RECORD_DECL¬      ( TYPE_DECL );¶    elsif¬ TYPE_SPEC.TY = DN_ARRAY¬¬then  CODE_UNCONSTRAINED_ARRAY_DE
CL( TYPE_DECL );¶    elsif¬ TYPE_SPEC.TY = DN_ACCESS¬¬then  CODE_ACCESS_DECL¬      ( TYPE_DECL );¶¶    elsif¬ TYPE_SPEC.TY = DN_CONSTRAINED_ARRAY¬then
 CODE_CONSTRAINED_ARRAY_DECL  ( TYPE_DECL );¶¶¬¬¬¬¬-- PRIVATE / LIMITED PRIVATE TYPES¶¬¬¬¬¬-- Nothing to generate here; the full¶¬¬¬¬¬-- type declaration
¬in the private part¶¬¬¬¬¬-- will be processed normally.¶¶    elsif¬ TYPE_SPEC.TY = DN_PRIVATE¶    or¬ TYPE_SPEC.TY = DN_L_PRIVATE¶    then¶      if¬CO
DI.DEBUG  then¶¬PUT_LINE(¬"; CODE_TYPE_DECL : skip PRIVATE "¶¬¬& PRINT_NAME( D( LX_SYMREP, TYPE_NAME )¬)¶¬¬& " (deferred to full type)" );¶      end¬i
f;¶¶¬¬¬¬¬-- INCOMPLETE TYPES¶¬¬¬¬¬-- Same: deferred to full type.¶¶    elsif¬ TYPE_SPEC.TY = DN_INCOMPLETE¶    then¶      if¬CODI.DEBUG  then¶¬PUT_LINE(
¬"; CODE_TYPE_DECL : skip INCOMPLETE "¶¬¬& PRINT_NAME( D( LX_SYMREP, TYPE_NAME )¬)¶¬¬& " (deferred to full type)" );¶      end¬if;¶¶    else¶      PUT
_LINE( "; CODE_GEN.DECLARATIONS.CODE_TYPE_DECL : TYPE_SPEC.TY ("¶¬¬& NODE_NAME'IMAGE( TYPE_SPEC.TY ) & " NON FAIT POUR ) "¶¬¬& PRINT_NAME( D( LX_SYMRE
P, TYPE_NAME )¬)¶¬        );¶    end if;¶¶  end¬  CODE_TYPE_DECL;¶¬--==============--¶¶¶¶¬¬¬----------------------¶  procedure¬¬CODE_ENUMERATION_DECL¬(
 TYPE_DECL :TREE )¶  is¬¬¬----------------------¶¶    TYPE_ID¬: TREE¬¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶    TYPE_SPEC¬: TREE¬¬:= D( SM_TYPE_SPEC,¬TYPE
_ID );¶    TYPE_STR¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, TYPE_ID ) );¶    MAX_REP¬: INTEGER¬¬:= INTEGER'FIRST;¶    MIN_REP¬: INTEGER¬¬:= INTE
GER'LAST;¶¬¬-------------------¶    procedure¬CODE_ENUM_LITERAL_S¬¬( ENUM_LITERAL_S :TREE )¶    is¶      ENUM_LITERAL_SEQ¬: SEQ_TYPE¬:= LIST (¬ENUM_LI
TERAL_S );¶      ENUM_LITERAL_ID¬: TREE;¶      LAST_LITERAL¬¬: TREE;¶    begin¶      while  not IS_EMPTY( ENUM_LITERAL_SEQ )  loop¶        POP( ENUM_L
ITERAL_SEQ, ENUM_LITERAL_ID );¶        declare¶¬ENUM_ID_STR¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, ENUM_LITERAL_ID ) );¶¬REP¬¬: INTEGER¬¬:= DI(
 SM_REP, ENUM_LITERAL_ID );¶        begin¶¬if  MIN_REP > REP  then MIN_REP := REP;¬end if;¶¬if  MAX_REP < REP  then MAX_REP := REP;¬end if;¶¬if  ENUM_
ID_STR /= "'""'"¶¬then  PUT_LINE( "db" & INTEGER'IMAGE( REP ) & ","¬& INTEGER'IMAGE( ENUM_ID_STR'LENGTH ) &¬", """ & ENUM_ID_STR & """" );¶¬else  PUT_
LINE( "db" & INTEGER'IMAGE( REP ) & ","¬& INTEGER'IMAGE( ENUM_ID_STR'LENGTH ) &¬", ""'""""'""" );¶¬end if;¶        end;¶        LAST_LITERAL := ENUM_L
ITERAL_ID;¶      end¬loop;¶      DI(¬CD_LAST, ENUM_LITERAL_S, DI (¬SM_REP, LAST_LITERAL ) );¶¶  end¬CODE_ENUM_LITERAL_S;¶¬-------------------¶¶  begin
¶    DI( CD_LEVEL,¬  TYPE_SPEC, INTEGER( CODI.CUR_LEVEL ) );¶    DB( CD_COMPILED,  TYPE_SPEC, TRUE );¶¶    PUT_LINE( TYPE_STR & " = '" & TYPE_STR & "'
" );¶    PUT( "namespace " & TYPE_STR );¶    if  CODI.DEBUG¬then PUT(¬tab50 & "; " & TYPE_STR & " ENUMERATION TYPE INFO" ); end if;¶    NEW_LINE;¶
PUT_LINE( "BEGIN_BLOC_DEF" );¶    CODE_ENUM_LITERAL_S( D( SM_LITERAL_S, TYPE_SPEC ) );¶    PUT_LINE( "END_BLOC_DEF" );¶    PUT_LINE( "IMAGES" & ASCII.
HT & "BYTES_BLOC" );¶    PUT_LINE( "CST " & "LST, d," & INTEGER'IMAGE(¬MAX_REP )¬);¶    PUT_LINE( "CST " & "FST, d," & INTEGER'IMAGE(¬MIN_REP )¬);¶
 PUT¬  ( "CST " & "SIZ, d," & INTEGER'IMAGE(¬DI( CD_IMPL_SIZE, TYPE_SPEC )¬) );¶    if  CODI.DEBUG¬then PUT(¬ASCII.HT & "; SIZ en bits !" ); end if;¶
   NEW_LINE;¶    PUT_LINE( "postpone" );¶    PUT_LINE( "  align_q" );¬¬¬¬¬¬¬¬¬¬-- Assurer  l'alignement de dpart de tout le bloc¶    PUT_LINE( "end po
stpone" );¶    PUT_LINE( "end namespace");¶    if  CODI.DEBUG¬then NEW_LINE; end if;¶¶  end¬CODE_ENUMERATION_DECL;¶¬----------------------¶¶¶¶¬¬¬-------
----------¶  procedure¬¬CODE_INTEGER_DECL¬¬( TYPE_DECL :TREE )¶  is¬¬¬-------------------¶¶    TYPE_ID¬¬: TREE¬¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶
TYPE_STR¬¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, TYPE_ID ) );¶    INTEGER_SPEC¬¬: TREE¬¬:= D( SM_TYPE_SPEC,¬TYPE_ID );¶    INT_RANGE¬¬: TREE¬¬:
= D( SM_RANGE, INTEGER_SPEC );¶    EXP_FST¬¬: TREE¬¬:= D( AS_EXP1, INT_RANGE );¶    EXP_LST¬¬: TREE¬¬:= D( AS_EXP2, INT_RANGE );¶    LVL_STR¬¬:constan
t¬STRING¬:= IMAGE(¬CODI.CUR_LEVEL );¶    SIZE_CHAR¬¬: CHARACTER¬:= OPER_SIZ_CHAR( INTEGER_SPEC );¶  begin¶    DI( CD_LEVEL,¬  INTEGER_SPEC, INTEGER( C
ODI.CUR_LEVEL¬) );¶    DB( CD_COMPILED,  INTEGER_SPEC, TRUE );¶¶    PUT_LINE( TYPE_STR & " = '" & TYPE_STR & "'" );¶    PUT( "namespace " & TYPE_STR )
;¶    if  CODI.DEBUG¬then  PUT( tab50 & "; " & TYPE_STR & " TYPE RANGE INFO" ); end if;¶    NEW_LINE;¶    PUT_LINE( "VAR SIZ, d" );¶    PUT_LINE( "VAR
 FST, " & SIZE_CHAR );¶    PUT_LINE( "VAR LST, " & SIZE_CHAR );¶¶    PUT_LINE( tab &¬"LI" & tab & IMAGE(¬DI( CD_IMPL_SIZE, INTEGER_SPEC ) ) );¶    PUT
_LINE( tab &¬"Sb" & tab & LVL_STR & ", SIZ" );¶¶    EXPRESSIONS.CODE_EXP( EXP_FST );¶    PUT_LINE( tab &¬'S' & SIZE_CHAR & tab & LVL_STR & ", FST" );¶
¶    EXPRESSIONS.CODE_EXP( EXP_LST );¶    PUT_LINE( tab &¬'S' & SIZE_CHAR & tab & LVL_STR & ", LST" );¶¶    PUT_LINE( "end namespace"¬);¶    if  CODI.
DEBUG¬then NEW_LINE; end if;¶¶  end¬CODE_INTEGER_DECL;¶¬-----------------¶¶¶¶¬¬¬-------------------¶  procedure¬¬CODE_FIXED_DECL¬¬( TYPE_DECL :TREE )¶  is
¬¬¬-----------------¶  begin¶    null;¶  end¬CODE_FIXED_DECL;¶¬-----------------¶¶¶¶¬¬¬-------------------¶  procedure¬¬CODE_FLOAT_DECL¬¬( TYPE_DECL :TREE )¶
 is¬¬¬-----------------¶¶    TYPE_ID¬¬: TREE¬¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶    TYPE_STR¬¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, TYPE_ID ) )
;¶    FLOAT_SPEC¬: TREE¬¬:= D( SM_TYPE_SPEC,¬TYPE_ID );¶  begin¶    DI( CD_LEVEL,¬  FLOAT_SPEC, INTEGER( CODI.CUR_LEVEL )¬);¶    DB( CD_COMPILED,  FLO
AT_SPEC, TRUE );¶¶    PUT_LINE( TYPE_STR & " = '" & TYPE_STR & "'" );¶    PUT( "namespace " & TYPE_STR );¶    if  CODI.DEBUG¬then  PUT( tab50 & "; " &
 TYPE_STR & " FLOAT TYPE INFO" ); end if;¶    NEW_LINE;¶    PUT_LINE( "CST SIZ, d," &¬INTEGER'IMAGE( DI( CD_IMPL_SIZE, FLOAT_SPEC ) ) );¶    PUT_LINE(

 "end namespace"¬);¶    if  CODI.DEBUG¬then NEW_LINE; end if;¶¶  end¬CODE_FLOAT_DECL;¶¬---------------¶¶¶¶¬¬¬-----------------¶  procedure¬¬CODE_RECORD
_DECL¬¬( TYPE_DECL :TREE )¶  is¬¬¬-----------------¶¶    TYPE_ID¬¬: TREE¬¬¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶    TYPE_SPEC¬¬: TREE¬¬¬:= D( SM_TYPE_SPE
C,¬TYPE_ID );¶    TYPE_ID_STR¬¬:constant¬STRING¬¬:= PRINT_NAME( D( LX_SYMREP, TYPE_ID ) );¶    LVL¬¬¬: LEVEL_NUM¬¬renames CODI.CUR_LEVEL;¶    LVL_STR¬
¬:constant¬STRING¬¬:= IMAGE(¬LVL );¶    IS_STATIC¬¬: BOOLEAN¬¬¬:= TRUE;¶    STATIC_SIZE¬¬: NATURAL¬¬¬:= 0;¶¶  begin¶¬¬¬¬¬-- Resolve private/incomplete¬
to full type¶    if  TYPE_SPEC.TY = DN_L_PRIVATE¶    or  TYPE_SPEC.TY = DN_PRIVATE¶    then¶      TYPE_SPEC := D( SM_TYPE_SPEC, TYPE_SPEC );¶    elsif
¬ TYPE_SPEC.TY = DN_INCOMPLETE¬ then¶      TYPE_SPEC := D( XD_FULL_TYPE_SPEC, TYPE_SPEC );¶    end if;¶    DI( CD_LEVEL,¬  TYPE_SPEC, INTEGER( CODI.CU
R_LEVEL ) );¶    DB( CD_COMPILED,  TYPE_SPEC, TRUE );¶¶    PUT_LINE( TYPE_ID_STR & " = '" & TYPE_ID_STR & "'" );¶    PUT( "namespace " & TYPE_ID_STR )
;¶    if  CODI.DEBUG¬then PUT(¬tab50 & "; " & TYPE_ID_STR & " RECORD TYPE INFO" ); end if;¶    NEW_LINE;¶    PUT_LINE( "VAR SIZ, d" );¶¶¬¬¬------------
-------------¶¬¬¬INSERE_LES_DISCRIMINANTS:¶    declare¶      DSCRMT_DECL_S¬¬: SEQ_TYPE¬:= LIST( D( AS_DSCRMT_DECL_S,¬TYPE_DECL¬) );¶      DSCRMT_DECL¬
¬: TREE;¶    begin¶      while  not IS_EMPTY( DSCRMT_DECL_S )  loop¶        POP( DSCRMT_DECL_S, DSCRMT_DECL¬);¶        declare¶¬DISCRIMINANT_ID_S¬: SE
Q_TYPE¬:= LIST( D( AS_SOURCE_NAME_S,¬DSCRMT_DECL ) );¶¬DISCRIMINANT_ID¬: TREE;¶        begin¶¬while  not IS_EMPTY( DISCRIMINANT_ID_S )  loop¶¬  POP( D
ISCRIMINANT_ID_S, DISCRIMINANT_ID );¶¬  PUT_LINE( "USEINFO " & PRINT_NAME( D(¬LX_SYMREP, DISCRIMINANT_ID ) ) );¶¬end loop;¶        end;¶      end¬loop
;¶    end¬INSERE_LES_DISCRIMINANTS;¶¬-----------------------¶¶¬¬¬-------------------¶¬¬¬INSERE_LES_CHAMPS:¶    declare¶      V_DECL_S¬¬: SEQ_TYPE¬:= LI
ST( D( AS_DECL_S, D( SM_COMP_LIST,¬TYPE_SPEC¬) ) );¶      V_DECL¬¬: TREE;¶    begin¶      while  not IS_EMPTY( V_DECL_S )  loop¶        POP( V_DECL_S,
 V_DECL¬);¶¶        declare¶¬COMP_ID_S¬¬: SEQ_TYPE¬:= LIST( D( AS_SOURCE_NAME_S,¬V_DECL ) );¶¬COMP_ID¬¬: TREE;¶¶        begin¶¬while  not IS_EMPTY( CO
MP_ID_S )  loop¶¬  POP( COMP_ID_S, COMP_ID );¶¶¬  declare¶¬    COMP_TYPE¬: TREE¬¬:= D( SM_OBJ_TYPE, COMP_ID );¶¬    COMP_TYPE_NAME¬: TREE¬¬:= D( XD_SO
URCE_NAME, COMP_TYPE );¶¬    COMP_TYPE_STR¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, COMP_TYPE_NAME ) );¶¬    COMP_ID_STR¬:constant¬STRING¬:= PRIN
T_NAME( D( LX_SYMREP, COMP_ID ) );¶¶¬  begin¶¬    if  COMP_TYPE.TY = DN_CONSTRAINED_ARRAY  then¶¬      if¬not DB( CD_COMPILED, COMP_TYPE )  then¶¬
   PUT_LINE( COMP_ID_STR¬& " = '" & COMP_ID_STR & "'" );¶¬        PUT_LINE( " namespace " & COMP_TYPE_STR );¶¬        PROCESS_CONSTRAINED_ARRAY_TYPE_
SPEC( COMP_TYPE );¶¬      end¬if;¶¶¬      PUT( "USEINFO " & LVL_STR & ", " & COMP_ID_STR & ", "¬);¶¬      PUT( tab & "LVA" & tab & LVL_STR & ", " );¶¬
      REGIONS_PATH(¬D( XD_SOURCE_NAME, D( SM_TYPE_SPEC, COMP_TYPE_NAME ) ) );¶¶¬      PUT_LINE(  COMP_TYPE_STR & ".SIZ"¬);¶¶¬    else¶¬      PUT( "USE
INFO " & LVL_STR & ", " & COMP_ID_STR & ", "¬);¶¬      if¬COMP_TYPE.TY = DN_ENUMERATION¬ then¬¬¬¬¬¬¬¬-- LES ENUM INFOS SONT EN CONSTANTES¶¬        PUT(
 tab & "LCA" & tab );¶¬      else¶¬        PUT( tab & "LVA" & tab & IMAGE(¬DI( CD_LEVEL, COMP_TYPE ) ) &¬", " );¶¬      end¬if;¶¬      REGIONS_PATH(¬C
OMP_TYPE_NAME );¶¬      PUT_LINE( PRINT_NAME( D( LX_SYMREP, COMP_TYPE_NAME ) )  & ".SIZ" );¶¬    end if;¶¶¬    if  COMP_TYPE.TY in CLASS_SCALAR  or  C
OMP_TYPE.TY in CLASS_CONSTRAINED  then¶¬      if¬D( CD_IMPL_SIZE, COMP_TYPE ) = TREE_VOID  then¶¬        IS_STATIC := FALSE;¶¬      end¬if;¶¬    elsif
¬ COMP_TYPE.TY = DN_RECORD  then¶¬      if¬not IS_EMPTY( LIST(¬D( SM_DISCRIMINANT_S, COMP_TYPE ) ) )  then¶¬        IS_STATIC := FALSE;¶¬      end¬if;
¶¬    else¶¬      IS_STATIC := FALSE;¶¬    end if;¶¬  end;¶¬end loop;¶        end;¶      end¬loop;¶    end¬INSERE_LES_CHAMPS;¶¬-----------------¶¶¬¬¬-
----------------------¶¬¬¬TRAITER_LES_DISCRIMINANTS:¶    declare¶      DSCRMT_DECL_S¬¬: SEQ_TYPE¬:= LIST( D( AS_DSCRMT_DECL_S,¬TYPE_DECL¬) );¶
DSCRMT_DECL¬¬: TREE;¶    begin¶      while  not IS_EMPTY( DSCRMT_DECL_S )  loop¶        POP( DSCRMT_DECL_S, DSCRMT_DECL¬);¶        declare¶¬DISCRIMINA
NT_ID_S¬: SEQ_TYPE¬:= LIST( D( AS_SOURCE_NAME_S,¬DSCRMT_DECL ) );¶¬DISCR_TYPE_DEFN¬: TREE¬¬:= D( SM_DEFN, D( AS_NAME, DSCRMT_DECL ) );¶¬DISCR_TYPE_SPE
C¬: TREE¬¬:= D( SM_TYPE_SPEC,¬DISCR_TYPE_DEFN );¶¬DISCRIMINANT_ID¬: TREE;¶¬SIZE_CHAR¬¬: CHARACTER¬:= 'x';¶        begin¶¬if  TYPE_SPEC.TY = DN_INTEGER
¬ then¶¬  SIZE_CHAR := OPER_SIZ_CHAR(¬TYPE_SPEC¬);¶¬end if;¶¬while  not IS_EMPTY( DISCRIMINANT_ID_S )  loop¶¬  POP( DISCRIMINANT_ID_S, DISCRIMINANT_ID
 );¶¶¬  PUT( tab & "LVA" & tab & IMAGE( DI( CD_LEVEL, DISCR_TYPE_SPEC ) ) &¬", " );¶¬  REGIONS_PATH( DISCR_TYPE_DEFN );¶¬  PUT_LINE( PRINT_NAME( D( LX
_SYMREP, D( XD_SOURCE_NAME, DISCR_TYPE_SPEC ) ) )¬& ".SIZ" );¶¬  PUT_LINE( tab & "Sa" & tab & LVL_STR & ", " & PRINT_NAME(¬D( LX_SYMREP, DISCRIMINANT_
ID¬) ) & "__u" );¶¶¬  PUT_LINE( tab & "LI 0" & tab & " ; offset a faire" );¶¬  PUT_LINE( tab & "Sd" & tab & LVL_STR & ", " & PRINT_NAME(¬D( LX_SYMREP,
 DISCRIMINANT_ID¬) ) & "__o" );¶¶¬end loop;¶        end;¶      end¬loop;¶    end¬TRAITER_LES_DISCRIMINANTS;¶¬-----------------------¶¶¬¬¬-------------
-------¶¬¬¬TRAITER_LES_CHAMPS:¶    declare¶      V_DECL_S¬¬: SEQ_TYPE¬:= LIST( D( AS_DECL_S, D( SM_COMP_LIST,¬TYPE_SPEC¬) ) );¶      V_DECL¬¬: TREE;¶
   begin¶¶      if¬IS_STATIC¬ then¶        PUT_LINE( "virtual at 0" );¶      end¬if;¶¶      while  not IS_EMPTY( V_DECL_S )  loop¶        POP( V_DECL_
S, V_DECL¬);¶        declare¶--¬FIELD_TYPE_DEF¬: TREE¬¬:= D( AS_TYPE_DEF, V_DECL );¶--¬FIELD_TYPE_NAME¬: TREE¬¬:= D( AS_NAME, FIELD_TYPE_DEF¬);¶--¬FIE
LD_TYPE_DEFN¬: TREE¬¬:= D( SM_DEFN, FIELD_TYPE_NAME );¶--¬FIELD_TYPE_SPEC¬: TREE¬¬:= D( SM_TYPE_SPEC,¬FIELD_TYPE_DEFN );¶¶¬COMP_ID_S¬¬: SEQ_TYPE¬:= LI
ST( D( AS_SOURCE_NAME_S,¬V_DECL ) );¶¬COMP_ID¬¬: TREE;¶¶        begin¶¬while  not IS_EMPTY( COMP_ID_S )  loop¶¬  POP( COMP_ID_S, COMP_ID );¶¶¬  declar
e¶¬    COMP_TYPE¬: TREE¬¬:= D( SM_OBJ_TYPE, COMP_ID );¶¬    COMP_TYPE_NAME¬: TREE¬¬:= D( XD_SOURCE_NAME, COMP_TYPE );¶¬    COMP_TYPE_STR¬:constant¬STR
ING¬:= PRINT_NAME( D( LX_SYMREP, COMP_TYPE_NAME ) );¶¬    COMP_ID_STR¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, COMP_ID ) );¶¬    COMP_SIZE¬: NATU
RAL;¶¶¬  begin¶¬    if  IS_STATIC  then¶¬      COMP_SIZE := DI( CD_IMPL_SIZE, COMP_TYPE );¶¬      if¬COMP_SIZE¬< CODI.STORAGE_UNIT¬ then¶¬        COMP
_SIZE := CODI.STORAGE_UNIT;¶¬      end¬if;¶¬      PUT_LINE( "STATOFS " & COMP_ID_STR¶¬¬    & ','¬& INTEGER'IMAGE( COMP_SIZE / CODI.STORAGE_UNIT ) );¶¬
     STATIC_SIZE := STATIC_SIZE + COMP_SIZE;¶¬    else¶¬      PUT_LINE( "; OFFSET NON STATIQUE A FAIRE" );¶¬    end if;¶¬  end;¶¬end loop;¶        en
d;¶      end¬loop;¶¶      if¬IS_STATIC¬ then¶        PUT_LINE( "size = $" );¶        PUT_LINE( "end virtual" );¶        DI( CD_IMPL_SIZE, TYPE_SPEC, S
TATIC_SIZE );¶      end¬if;¶¶    end¬TRAITER_LES_CHAMPS;¶¬-----------------¶¶    PUT_LINE( "end namespace"¬);¶    if  CODI.DEBUG¬then  NEW_LINE; end¬
if;¶¶  end¬CODE_RECORD_DECL;¶¬-----------------¶¶¶¶¬¬¬-----------------------------¶  procedure¬¬CODE_UNCONSTRAINED_ARRAY_DECL¬¬( TYPE_DECL :TREE )¶  i
s¬¬¬-----------------------------¶¶    TYPE_ID¬¬: TREE¬¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶    TYPE_ID_STR¬¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMR

EP, TYPE_ID ) );¶    TYPE_SPEC¬¬: TREE¬¬:= D( SM_TYPE_SPEC,¬TYPE_ID );¶    INDEX_SUBTYPE_S¬¬: SEQ_TYPE¬:= LIST( D( SM_INDEX_S, TYPE_SPEC ) );¶    DIM_
NBR¬¬: NATURAL¬¬:= 1;¶    TOTAL_DIMS¬¬: NATURAL¬¬:= 0;¶    LVL¬¬¬: LEVEL_NUM¬renames CODI.CUR_LEVEL;¶    LVL_STR¬¬:constant¬STRING¬:= IMAGE(¬LVL );¶¶¬
¬---------------¶    procedure¬USEINFO_OFFSETS¬( IDX_TYPE_LIST :in¬out SEQ_TYPE )¶    is¶      IDX_TYPE¬¬: TREE;¶      DIM_NBR_STR¬¬:constant¬STRING¬:
= IMAGE(¬DIM_NBR );¶    begin¶      POP( IDX_TYPE_LIST, IDX_TYPE );¶¶      declare¶        IDX_TYPE_SPEC¬: TREE¬¬:= D( SM_TYPE_SPEC,¬IDX_TYPE );¶
 IDX_TYPE_NAME¬: TREE¬¬:= D( AS_NAME, IDX_TYPE );¶        IDX_TYPE_DEFN¬: TREE¬¬:= D( SM_DEFN, IDX_TYPE_NAME );¶¶      begin¶        TOTAL_DIMS := T
OTAL_DIMS + 1;¶¶        if  not IS_EMPTY( IDX_TYPE_LIST¬)  then¶¬DIM_NBR := DIM_NBR + 1;¶¬USEINFO_OFFSETS( IDX_TYPE_LIST );¶¶¬PUT_LINE(¬"SIZ_" & DIM_N
BR_STR & " = $"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶¬PUT_LINE(¬"FST_" & DIM_NBR_STR & " = $"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶¬PUT_LINE(¬"LST_" & DIM_NBR_STR
 & " = $"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶        else¶¬PUT_LINE(¬"COMP_SIZ = $" );¶¬PUT_LINE(¬tab & "rd 1 " );¶¬PUT_LINE(¬"FST_" & DIM_NBR_STR & " = $
"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶¬PUT_LINE(¬"LST_" & DIM_NBR_STR & " = $"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶¶--¬PUT_LINE(¬"VAR NDIMS, b" );¶--¬PUT_LINE(¬
tab & "LI" & tab & IMAGE( TOTAL_DIMS ) );¶--¬PUT_LINE(¬tab & "Sb" & tab & LVL_STR & ", NDIMS" );¶¶--¬PUT_LINE(¬"USEINFO COMP" );¶¶-- ELT__u¬vers TYPE¬
DU TYPE ELEMENT¶-- D( SM_COMP_TYPE,¬TYPE_SPEC¬);¶¶        end if;¶¶--¬PUT( "USEINFO " & LVL_STR & ", DIM_" & DIM_NBR_STR & ", " );¶¶--¬if  IDX_TYPE_SP
EC.TY = DN_ENUMERATION  then¬¬¬¬¬¬¬¬-- LES ENUM INFOS SONT EN CONSTANTES¶--¬PUT( tab & "LCA" & tab );¶--¬else¶--¬PUT( tab & "LVA" & tab & IMAGE( DI( CD
_LEVEL, IDX_TYPE_SPEC¬) ) & ", " );¶--¬end if;¶--¬REGIONS_PATH( IDX_TYPE_DEFN );¶--¬PUT_LINE(¬PRINT_NAME( D( LX_SYMREP, IDX_TYPE_NAME¬) )  & ".SIZ" );
¶¶--¬PUT_LINE(¬tab & "Sa" & tab & LVL_STR & ", DIM_" &¬DIM_NBR_STR & "__u"¬);¶      end;¶    end¬USEINFO_OFFSETS;¶¬---------------¶¶  begin¶    DI( CD
_LEVEL,¬  TYPE_SPEC, INTEGER( CODI.CUR_LEVEL ) );¶    DB( CD_COMPILED,  TYPE_SPEC, TRUE );¶¶    PUT_LINE( TYPE_ID_STR & " = '" & TYPE_ID_STR & "'" );¶
   PUT( "namespace " & TYPE_ID_STR );¶    if  CODI.DEBUG¬then PUT(¬tab50 & "; " & TYPE_ID_STR & " UNCONSTRAINED ARRAY SUBTYPE INFO" ); end if;¶    NE
W_LINE;¶    PUT_LINE( "VAR SIZ, d" );¶    PUT_LINE( "  virtual at 4" );¬¬¬¬¬¬¬¬¬¬-- Commence apres SIZ¶¶    USEINFO_OFFSETS( INDEX_SUBTYPE_S );¶¶    PU
T_LINE( "  end virtual"¬);¬¬¬¬¬¬¬¬¬¬-- Commence apres SIZ¶¶    PUT_LINE( "end namespace"¬);¶    if  CODI.DEBUG¬then NEW_LINE; end if;¶¶  end¬CODE_UNCON
STRAINED_ARRAY_DECL;¶¬-----------------------------¶¶¶¶¬¬¬--==================================--¶  procedure¬¬  PROCESS_CONSTRAINED_ARRAY_TYPE_SPEC¬¬
( TYPE_SPEC :TREE )¶  is¬¬¬--==================================--¶¶    DIM_NBR¬¬: NATURAL¬¬¬:= 0;¶    LVL¬¬¬: LEVEL_NUM¬¬renames CODI.CUR_LEVEL;¶
LVL_STR¬¬:constant¬STRING¬¬:= IMAGE(¬CODI.CUR_LEVEL );¶    TOTAL_ELEMENTS¬¬: NATURAL;¶¶    BASE_TYPE¬¬: TREE¬¬¬:= D( SM_BASE_TYPE,¬TYPE_SPEC¬);¶    CO
MP_TYPE¬¬: TREE¬¬¬:= D( SM_COMP_TYPE,¬BASE_TYPE¬);¶    COMP_SIZE_TREE¬¬: TREE¬¬¬:= D( CD_IMPL_SIZE,¬COMP_TYPE¬);¶    IS_STATIC¬¬: BOOLEAN¬¬¬:= COMP_SI
```

```
     ZE_TREE /= TREE_VOID;¶    ARRAY_STATIC_SIZE¬: NATURAL¬¬¬:= 0;¶¶¬¬------------------------------¶    procedure¬COMPILE_ARRAY_TYPE_DIMENSION¬¬( IDX_TYPE_L
     IST :in¬out SEQ_TYPE )¶    is¶      IDX_TYPE¬¬: TREE;¶      DIM_NBR_STR¬¬:constant¬STRING¬:= IMAGE(¬DIM_NBR+1¬);¶    begin¶      POP( IDX_TYPE_LIST, I
     DX_TYPE );¶      DIM_NBR := DIM_NBR + 1;¶¶      if¬IS_EMPTY(¬IDX_TYPE_LIST )  then¶      declare¶¬ELEMENT_SIZ¬¬: NATURAL¬¬:= DI( CD_IMPL_SIZE, COMP_
     TYPE );¬¬¬¬-- TAILLE¬EN BITS¶¬ELEMENT_SIZ_STR¬¬:constant¬STRING¬:= IMAGE(¬ELEMENT_SIZ / 8 );¬¬¬¬-- IMAGE DE TAILLE EN OCTETS¶      begin¶¬ARRAY_STATI
     C_SIZE := ELEMENT_SIZ;¶¬PUT_LINE(¬"VAR COMP_SIZ, d" );¶¬PUT_LINE(¬"VAR FST_" & DIM_NBR_STR & ", d" );¶¬PUT_LINE(¬"VAR LST_" & DIM_NBR_STR & ", d" );¶¶
     ¬PUT_LINE(¬tab & "LI" & tab & ELEMENT_SIZ_STR );¬¬¬¬¬¬¬¬-- TAILLE¬D'UN ELEMENT DU TABLEAU¶¬PUT_LINE(¬tab & "Sd" & tab & LVL_STR & ", COMP_SIZ" );¬¬¬¬¬¬
     ¬-- DWORD COMP_SIZ¶¬PUT_LINE(¬tab & "Ld" & tab & LVL_STR & ", COMP_SIZ" );¬¬¬¬¬¬¬-- recharge pour MUL suivant¶      end;¶¶      else¶      COMPILE
     _ARRAY_TYPE_DIMENSION( IDX_TYPE_LIST );¶¶      PUT_LINE( "VAR SIZ_" & DIM_NBR_STR & ", d" );¶      PUT_LINE( "VAR FST_" & DIM_NBR_STR & ", d" );¶
0130       PUT_LINE( "VAR LST_" & DIM_NBR_STR & ", d" );¶¶      PUT_LINE( tab & "MUL"¬);¶      PUT_LINE( tab & "Sd" & tab & LVL_STR & ", SIZ_" & DIM_N
     BR_STR¬);¬¬¬¬¬¬-- METTRE¬LA TAILLE¬TRANCHE A¬CELLE LAISSEE PAR LE CALCUL SUR LA DIM PRECEDENTE¶      PUT_LINE( tab & "Ld" & tab & LVL_STR & ", SIZ_"
      & DIM_NBR_STR¬);¬¬¬¬¬¬-- recharge pour MUL suivant¶    end¬if;¶¶    if¬IDX_TYPE.TY = DN_INTEGER  then¶      declare¶¬IDX_RANGE¬¬: TREE¬¬:= D( S
     M_RANGE, IDX_TYPE );¶¬RANGE_FIRST¬: TREE¬¬:= D( AS_EXP1, IDX_RANGE );¶¬RANGE_LAST¬: TREE¬¬:= D( AS_EXP2, IDX_RANGE );¶      begin¶¬if  RANGE_FIRST.T
     Y /= DN_NUMERIC_LITERAL¶¬or  RANGE_LAST.TY /= DN_NUMERIC_LITERAL¶¬then¶¬  IS_STATIC := FALSE;¶¬end if;¶¶¬EXPRESSIONS.CODE_EXP( RANGE_FIRST );¶¬PUT_LIN
     E(¬tab & "Sd" & tab & LVL_STR & ", FST_" &¬DIM_NBR_STR );¶¬EXPRESSIONS.CODE_EXP( RANGE_LAST );¶¬PUT_LINE(¬tab & "Sd" & tab & LVL_STR & ", LST_" &¬DIM_
     NBR_STR );¶¶¬PUT_LINE(¬tab & "Ld" & tab & LVL_STR & ", LST_" &¬DIM_NBR_STR );¶¬PUT_LINE(¬tab & "INC" );¶¬PUT_LINE(¬tab & "Ld" & tab & LVL_STR & ", FST
     _" &¬DIM_NBR_STR );¶¬PUT_LINE(¬tab & "SUB" );¶¶¬if  IS_STATIC  then¶¬  ARRAY_STATIC_SIZE¬:= ( DI( SM_VALUE, RANGE_LAST¬) + 1 - DI( SM_VALUE, RANGE_FIR
     ST ) ) *¬ARRAY_STATIC_SIZE;¶¬end if;¶      end;¶    end¬if;¶¶    end¬COMPILE_ARRAY_TYPE_DIMENSION;¶¬----------¶¶¶¶¬----------
     --------¶    procedure¬COMPUTE_INFO_OFFSETS¬( IDX_TYPE_LIST :in¬out SEQ_TYPE )¶    is¶      IDX_TYPE¬¬: TREE;¶      DIM_NBR_STR¬¬:constant¬STRING¬:= I
0140 MAGE(¬DIM_NBR+1¬);¶    begin¶      POP( IDX_TYPE_LIST, IDX_TYPE );¶      DIM_NBR := DIM_NBR + 1;¶¶      if¬IS_EMPTY(¬IDX_TYPE_LIST )  then¶¬PUT_LINE(¬
     "COMP_SIZ = $" );¶¬PUT_LINE(¬tab & "rd 1 " );¶¬PUT_LINE(¬"FST_" & DIM_NBR_STR & " = $"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶¬PUT_LINE(¬"LST_" & DIM_NBR_STR
     & " = $"¬);¶¬PUT_LINE(¬tab & "rd 1 " );¶      else¶      COMPUTE_INFO_OFFSETS(¬IDX_TYPE_LIST );¶¶      PUT_LINE( "SIZ_" & DIM_NBR_STR & " = $" );¶
           PUT_LINE( tab & "rd 1 " );¶      PUT_LINE( "FST_" & DIM_NBR_STR & " = $" );¶      PUT_LINE( tab & "rd 1 " );¶      PUT_LINE( "LST_" & DI
     M_NBR_STR & " = $" );¶      PUT_LINE( tab & "rd 1 " );¶    end¬if;¶¶    end¬COMPUTE_INFO_OFFSETS;¶¬----------¶¶  begin¶    DI( CD_LEVEL,
      TYPE_SPEC, INTEGER( LVL ) );¶    PUT_LINE( "VAR SIZ, d" );¶    PUT_LINE( "  namespace info" );¶¶¬¬¬----------¶¬¬¬DESCRIPTOR_ON_STACK:¶    be
     gin¶      declare¶      IDX_TYPE_LIST¬: SEQ_TYPE¬:= LIST( D( SM_INDEX_SUBTYPE_S, TYPE_SPEC ) );¶      begin¶      COMPILE_ARRAY_TYPE_DIMENSION( ID
     X_TYPE_LIST );¶    end;¶      PUT_LINE( tab¬& "MUL" );¶      PUT_LINE( tab¬& "Sd" & tab & LVL_STR & ", SIZ" );¶¶      if¬IS_STATIC¶      then  DI( C
     D_IMPL_SIZE,¬TYPE_SPEC,  ARRAY_STATIC_SIZE¬);¶      end¬if;¶¶      PUT_LINE( "  end namespace" );¶¶      PUT_LINE( "  virtual at 4" );¬¬¬¬¬¬¬¬¬-- Comm
     ence apres SIZ¶      declare¶      IDX_TYPE_LIST¬: SEQ_TYPE¬:= LIST( D( SM_INDEX_SUBTYPE_S, TYPE_SPEC ) );¶      begin¶      DIM_NBR := 0;¶

0150  COMPUTE_INFO_OFFSETS(¬IDX_TYPE_LIST );¶      end;¶      PUT_LINE( "  end virtual" );¶¶      PUT_LINE( "end namespace" );¶      if¬CODI.DEBUG  then NE
     W_LINE; end if;¶¶    end¬DESCRIPTOR_ON_STACK;¶¬----------¶¶    DB( CD_COMPILED,¬TYPE_SPEC, TRUE );¶¶  end¬  PROCESS_CONSTRAINED_ARRAY_TYPE_SP
     EC;¶¬--======================================¶¶¶¶¬¬¬¬------------------------------¶  procedure¬¬CODE_CONSTRAINED_ARRAY_DECL¬¬( TYPE_DECL :TREE )¶  is¬¬¬¬-
     -------------------------¶¶    TYPE_NAME¬¬: TREE¬¬¬:= D( AS_SOURCE_NAME, TYPE_DECL );¶    TYPE_NAME_STR¬¬:constant¬STRING¬¬:= PRINT_NAME( D( LX_SYMREP
     , TYPE_NAME ) );¶    TYPE_SPEC¬¬: TREE¬¬¬:= D( SM_TYPE_SPEC,¬TYPE_NAME¬);¶¶  begin¶    PUT_LINE( TYPE_NAME_STR &¬" = '" & TYPE_NAME_STR & "'" );¶    P
     UT( "namespace " & TYPE_NAME_STR );¶    if  CODI.DEBUG¬then PUT(¬tab50 & "; array decl constrained array type info" ); end if;¶    NEW_LINE;¶¶    PROC
     ESS_CONSTRAINED_ARRAY_TYPE_SPEC( TYPE_SPEC );¶¶  end¬CODE_CONSTRAINED_ARRAY_DECL;¶¬----------¶¶¶¶¬¬¬¬----------¶  procedure¬¬¬C
     ODE_ACCESS_DECL¬¬( TYPE_DECL :TREE )¶  is¬¬¬¬----------¶  begin¶    null;¶  end¬CODE_ACCESS_DECL;¶¬----------¶¶¶¶¬¬¬¬--==========
     ¶  procedure¬¬¬  CODE_SUBTYPE_DECL¬¬( SUBTYPE_DECL :TREE )¶  is¬¬¬¬--==========¶¶    SUBTYPE_ID¬¬: TREE¬¬:= D( AS_SOURCE_NAME, SUBTYPE_DECL)
     ;¶    TYPE_SPEC¬¬: TREE¬¬:= D( SM_TYPE_SPEC,¬SUBTYPE_ID );¶    SUBTYPE_STR¬¬:constant¬STRING¬:= PRINT_NAME( D( LX_SYMREP, SUBTYPE_ID¬) );¶  begin¶
0160 DI( CD_LEVEL,¬  TYPE_SPEC, INTEGER( CODI.CUR_LEVEL ) );¶    DB( CD_COMPILED,  TYPE_SPEC, TRUE );¶¶    if  TYPE_SPEC.TY = DN_CONSTRAINED_ARRAY¶    then
     ¶      PUT_LINE( SUBTYPE_STR &¬" = '" & SUBTYPE_STR & "'" );¶      PUT( "namespace " & SUBTYPE_STR );¶      if¬CODI.DEBUG  then PUT( tab50 &¬"; " & SU
     BTYPE_STR & " CONSTRAINED ARRAY SUBTYPE INFO" ); end if;¶      NEW_LINE;¶¶      PROCESS_CONSTRAINED_ARRAY_TYPE_SPEC( TYPE_SPEC );¶¶    elsif¬ TYPE_SPE
     C.TY = DN_ENUMERATION¶    then¶      PUT_LINE( SUBTYPE_STR &¬" = '" & SUBTYPE_STR & "'" );¶      PUT( "namespace " & SUBTYPE_STR );¶      if¬CODI.DEBU
     G  then PUT( tab50 &¬"; " & SUBTYPE_STR & " ENUMERATION SUBTYPE INFO" ); end if;¶      NEW_LINE;¶      PUT_LINE( "VAR SIZ, d" );¶--      D( SM_BASE_TY
     PE, TYPE_SPEC );¶--      D( SM_RANGE, TYPE_SPEC );¶      PUT_LINE( "end namespace" );¶      if¬CODI.DEBUG  then NEW_LINE; end if;¶¶¶    elsif¬ TYPE_SP
     EC.TY = DN_INTEGER¶    then¶      PUT_LINE( SUBTYPE_STR &¬" = '" & SUBTYPE_STR & "'" );¶      PUT( "namespace " & SUBTYPE_STR );¶      if¬CODI.DEBUG
     then PUT( tab50 &¬"; " & SUBTYPE_STR & " INTEGER SUBTYPE INFO" ); end if;¶      NEW_LINE;¶      PUT_LINE( "VAR SIZ, d" );¶--      D( SM_BASE_TYPE, TYP
     E_SPEC );¶--      D( SM_RANGE, TYPE_SPEC );¶      PUT_LINE( "end namespace" );¶      if¬CODI.DEBUG  then NEW_LINE; end if;¶¶    else¶      PUT_LINE( "
     ;  CODE_SUBTYPE_DECL : TYPE_SPEC.TY PAS FAIT " & NODE_NAME'IMAGE( TYPE_SPEC.TY ) );¶    end if;¶¶  end¬  CODE_SUBTYPE_DECL;¶¬--==========--¶¶¶¬
0170 ----------¶end¬TYPES_DECLS;¶¬----------¶¶----------------------------------------------------------------------------------------------------
     --------------¶--¬1¬2¬3¬4¬5¬6¬7¬8¬9¬0¬1¬2¶
```